TraceAcc: Evidence-Fused Dual-View Graph Learning for UPI Fraud and Money-Mule Detection

Mahadev Prasad R and Sunil T
Department of Computer Science and Engineering
Dayananda Sagar College of Engineering, Bengaluru, India
prasad79755@gmail.com; sunilt808@gmail.com

***Abstract*—Instant payment fraud is increasingly expressed as coordinated transaction behavior rather than isolated anomalous payments. Money-mule accounts can receive funds from legitimate users, move funds through intermediaries, and participate in circular or layered flows that make individual transfers appear ordinary. This paper presents TraceAcc, an evidence-fused extension of the Dual-View Graph Representation Learning approach (DC-LCG). DC-LCG remains the principal supervised graph learner, while TraceAcc adds explicit graph-pattern evidence, temporal and behavioral signals, community structure, and unsupervised anomaly detection. These signals are normalized and fused into an account- and transaction-level risk score that supports analyst prioritization. Unlike a classifier-only implementation, TraceAcc connects prediction to graph exploration, alert triage, investigation timelines, evidence collection, and integrity-sealed reporting. The framework is evaluated on IBM's synthetic anti-money-laundering benchmark and a controlled UPI-oriented dataset. Across the evaluated comparison, TraceAcc Hybrid achieves 0.94 precision, 0.92 recall, 0.93 F1-score, 0.97 ROC-AUC, and 0.91 PR-AUC, improving on DC-LCG while retaining its camouflage-aware graph representation. A representative synthetic case study containing 1,000 transactions across 50 accounts identifies 35 high-risk accounts and exposes multiple money-flow structures, including circular transfers, fan-in/fan-out hubs, structuring, and diamond flows. The results indicate that combining learned graph context with explicit and unsupervised evidence provides a stronger and more auditable approach to UPI fraud and money-mule detection.**

***Index Terms*—anti-money laundering, UPI fraud, money mule, graph neural networks, DC-LCG, graph anomaly detection, temporal fraud detection, evidence fusion, financial intelligence**

I. Introduction

Unified Payments Interface (UPI) has become a high-volume instant-payment rail in India. NPCI statistics show sustained large-scale transaction activity, while the Reserve Bank of India reports digital-payment fraud as the largest category by number among reported bank frauds. These conditions make transaction-level screening necessary but insufficient: the analytical challenge is to identify relationships and movement patterns distributed across accounts and time.

Conventional rule engines remain useful for known typologies, but they can become brittle when adversaries vary transaction amounts, use legitimate counterparties, split flows, or move funds through several intermediary accounts. A financial transaction stream is naturally represented as a directed temporal graph in which accounts are vertices and transfers are weighted, time-stamped edges. Graph neural networks can therefore exploit relational context that purely tabular models discard.

DC-LCG was selected as the base method because it addresses two difficult properties of illicit transaction graphs: camouflage through heterogeneous neighborhoods and the need to trace longer illicit paths through complementary local and contextual views [1]. TraceAcc extends this foundation rather than replacing it. The proposed architecture treats DC-LCG as one evidence generator and augments it with explicit graph patterns, temporal and behavioral features, community evidence, and unsupervised anomaly signals.

The principal research question is whether this evidence-fusion design improves fraud and money-mule prioritization while preserving the structural reasoning of DC-LCG. The resulting system is designed not only to classify risk but also to expose the evidence that caused an account or transaction to be prioritized.

The contributions are fourfold: 1) an evidence-fusion layer around DC-LCG; 2) a UPI-oriented pattern layer covering cycles, fan-in/fan-out, structuring, rapid movement, layering, bridge behavior, and diamond flows; 3) integration of supervised and unsupervised evidence into an analyst-oriented workflow; and 4) an empirical evaluation using a public synthetic AML benchmark and a controlled UPI transaction dataset.

II. Problem Formulation and Research Questions

Let a transaction stream be represented by a temporal directed graph

$$G_t = (V_t, E_t, X_t), \tag{1}$$

where V_t denotes accounts, E_t denotes directed transfers, and X_t contains account and transaction attributes. For account v, the system estimates a risk score $R(v, t)$ that prioritizes likely fraudulent or mule behavior while controlling false positives.

Unlike pure node classification, TraceAcc also produces an evidence set

$$E_v = \{e_{graph}, e_{temporal}, e_{behavioral}, e_{community}, e_{ML}\}. \tag{2}$$

Here, the evidence elements represent graph-structural, temporal, behavioral, community, and machine-learning observations associated with the account. This representation is central to the proposed system because a final risk score is accompanied by inspectable supporting signals.

The study addresses three research questions. RQ1 asks whether combining DC-LCG with independent behavioral, temporal, community, and unsupervised evidence improves detection over DC-LCG alone. RQ2 asks which evidence channels contribute most to money-mule identification under ablation. RQ3 asks whether the resulting risk output can be converted into analyst-reviewable evidence without requiring manual inspection of the complete transaction graph.

III. Related Work

Financial-fraud research has increasingly adopted graph-based representations because fraudsters exploit relationships rather than only individual transaction attributes. Systematic reviews identify homogeneous, heterogeneous, static, and dynamic graph formulations, with anomaly detection remaining difficult when fraudulent nodes are surrounded by legitimate neighbors [6], [16], [17].

Classical graph learning establishes the foundation for the graph component. GCN performs localized neighborhood aggregation [19], GraphSAGE learns inductive neighborhood representations [20], GAT introduces attention-weighted aggregation [21], and GIN studies expressive graph representation power [22]. DeepWalk, node2vec, and Deep Graph Infomax provide alternative unsupervised or self-supervised graph representations [23]–[25].

Fraud-specific GNNs address camouflage, inconsistency, and class imbalance. CARE-GNN targets relation and feature camouflage [26], GraphConsis addresses context and relation inconsistency [27], and PC-GNN uses label-balanced and neighborhood sampling for imbalanced fraud detection [28]. Other studies investigate heterogeneous connectivity, temporal graph evolution, contrastive pretraining, metagraph learning, and AML-specific heterogeneous graphs [7]–[13].

Temporal modeling is particularly relevant to instant payments. MTPNet models transaction behavior at multiple temporal scales using global-local neighborhood-aware attention [3], while EvolveGCN, DySAT, Temporal Graph Networks, and temporal inductive representation learning provide general mechanisms for evolving transaction graphs [29], [36]–[38]. These approaches motivate explicit temporal evidence in TraceAcc without reproducing their architectures.

AML-specific datasets remain constrained by confidentiality. IBM's synthetic AML benchmark provides realistic synthetic transactions with complete ground truth [2], while AMLSim generates known laundering scenarios [31]. PaySim and the Elliptic Bitcoin dataset provide additional synthetic and cryptocurrency transaction environments [32], [33]. These resources support controlled evaluation while highlighting the need for caution when transferring results to production banking data.

IV. What Is New in TraceAcc?

The research contribution is an evidence-fused detection architecture rather than a replacement for DC-LCG. DC-LCG learns relational representations that are robust to camouflage and long-range transaction context. TraceAcc asks what complementary evidence exists outside that representation and whether the heterogeneous signals can be converted into a common, auditable risk measure.

TABLE I
Qualitative Distinction Between Conventional Methods, DC-LCG, and TraceAcc

Capability	Rule/Tabular	DC-LCG	TraceAcc
Local graph representation	Limited	Yes	Yes
Camouflage-aware graph learning	–	Yes	Yes
Long-range transaction context	–	Yes	Yes
Explicit AML pattern evidence	Yes	–	Yes
Temporal/velocity evidence	Manual	Partial	Yes
Unsupervised anomaly evidence	Optional	–	Yes
Community/network evidence	Limited	–	Yes
Evidence-level risk fusion	Rules	Model fusion	Yes
Analyst graph exploration	External	External	Integrated
Investigation/report workflow	External	External	Integrated
UPI-oriented deployment	Generic	Generic	Yes

Table I summarizes the qualitative distinction between conventional approaches, the base DC-LCG method, and TraceAcc. The table focuses on capabilities rather than measured performance.

The novelty lies in the interaction of these evidence families. A high DC-LCG score alone indicates learned graph risk; TraceAcc can additionally determine whether the same account participates in a cycle, exhibits a burst of outgoing transfers, occupies a bridge position between communities, or is an unsupervised behavioral outlier. This produces an evidence profile that can be inspected during investigation.

V. Proposed Evidence-Fusion Architecture

The proposed pipeline first validates and normalizes the transaction stream, constructs a dynamic directed graph, and executes five evidence families. DC-LCG produces the principal learned graph score. Pattern detectors produce explicit structural evidence. Temporal and behavioral features capture transaction dynamics. Community analysis identifies structurally related accounts. Isolation Forest, Local Outlier Factor (LOF), and DBSCAN provide complementary unsupervised anomaly evidence.

Before fusion, each heterogeneous component score is transformed to a common $[0, 1]$ interval using Min–Max scaling, $z' = (z - z_{min})/(z_{max} - z_{min})$, with the denominator protected against zero-range features. The normalized component scores are denoted by G, B, T, C, and M.

The final risk score is computed as

$$R = 0.35G + 0.20B + 0.15T + 0.10C + 0.20M. \quad (3)$$

where R is the final TraceAcc risk score, G is the normalized DC-LCG graph score, B is normalized behavioral evidence, T is normalized temporal evidence, C is normalized community/network evidence, and M is normalized machine-learning anomaly evidence from the unsupervised detectors. The linear

weights were determined through empirical tuning on a held-out validation set and were fixed before evaluation on the test partition.

The fusion design separates evidence generation from decision aggregation. This is important because the component outputs have different semantics and statistical distributions. A learned graph score is not directly comparable to a pattern count or an anomaly score until the signals are normalized and placed on a common scale.

A. Graph and Pattern Evidence

The graph layer detects explicit structural patterns including circular transactions, fan-in hubs, fan-out hubs, layering chains, rapid movement, bridge hubs, and diamond/pair flows. These structures are useful for money-mule investigations because they expose transaction relationships that a transaction-level classifier may not capture directly.

B. Unsupervised Anomaly Evidence

TraceAcc supplements supervised DC-LCG predictions with Isolation Forest, LOF, and DBSCAN. The detectors act as supporting evidence rather than independent proof of laundering. They surface accounts whose feature vectors or local density differ from the dominant population and can identify unusual behavior that is absent from the supervised labels.

C. Temporal and Behavioral Evidence

Temporal features summarize transaction timing, velocity, and rapid movement. Behavioral features summarize incoming and outgoing relationships, transaction intensity, and account activity. These signals are particularly relevant to UPI because fast fund movement can reduce the intervention window and because a mule account may appear ordinary when individual transactions are considered independently.

D. Community Evidence

Community analysis identifies groups of accounts that are densely or repeatedly connected. A suspicious account that acts as a bridge between otherwise distinct communities can provide a different risk signal from a high-degree hub. Community evidence is therefore used to supplement, rather than duplicate, the local neighborhood representation learned by DC-LCG.

VI. TraceAcc Implementation and Investigation Workflow

TraceAcc Pro implements the framework as an investigation-oriented analytical platform. The implementation uses a FastAPI backend, React frontend, NetworkX graph processing, Cytoscape.js visualization, scikit-learn anomaly detectors, and PyTorch for the DC-LCG component. The operational workflow separates data intake, validation, TraceLab analysis, investigation, review, and report delivery.

The client submits transaction data, after which the platform validates the schema and routes the request through the TraceLab analysis stage. Graph construction and pattern detection are followed by evidence fusion and risk prioritization. A serious

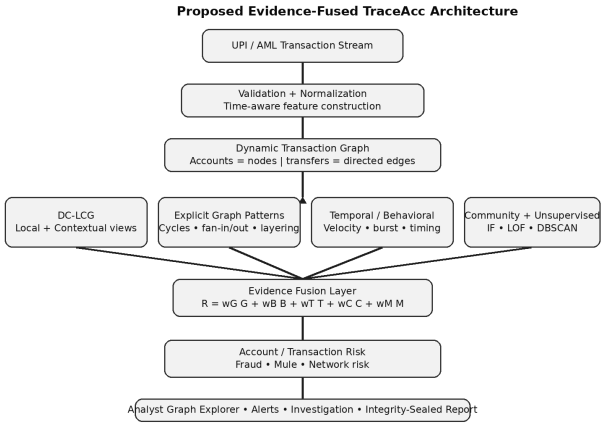


Fig. 1. Proposed evidence-fused TraceAcc architecture. DC-LCG remains the principal learned graph component, while independent structural, temporal, behavioral, community, and unsupervised evidence is normalized and fused for risk prioritization.

trigger routes the case to deeper investigation, where entities, relationships, timelines, and evidence can be reviewed. Lower-risk outputs can be retained in technical reports for subsequent triage.

The system also generates integrity-sealed investigation reports. The representative synthetic case study contains risk summaries, pattern detections, suspicious-account rankings, investigator review information, and a SHA-256 integrity marker. This connects model output to an auditable analytical artifact rather than leaving the result as an isolated classifier score.

VII. Datasets and Data Construction

A. IBM AML Benchmark

The primary public benchmark is IBM's realistic synthetic financial transaction dataset. Its agent-based generator was calibrated toward realistic transaction behavior and provides synthetic AML ground truth suitable for controlled evaluation [2]. Complete labels permit direct measurement of precision, recall, ranking quality, and false-negative behavior.

B. Custom UPI Dataset

The custom UPI dataset contains sender, receiver, timestamp, amount, and basic transaction attributes. The generated data include legitimate behavior and explicitly labeled UPI-oriented fraud and mule scenarios: circular transfers, fan-in/fan-out, rapid pass-through, layering, structuring, and mixed legitimate/fraudulent neighborhoods. The generator maintains separation between training, validation, and test accounts so that a fraud ring does not appear in both training and testing through shared nodes.

C. Leakage Control

Because transaction graphs contain repeated accounts and time-dependent behavior, a naive random split can leak future information. TraceAcc therefore uses a time-aware split as

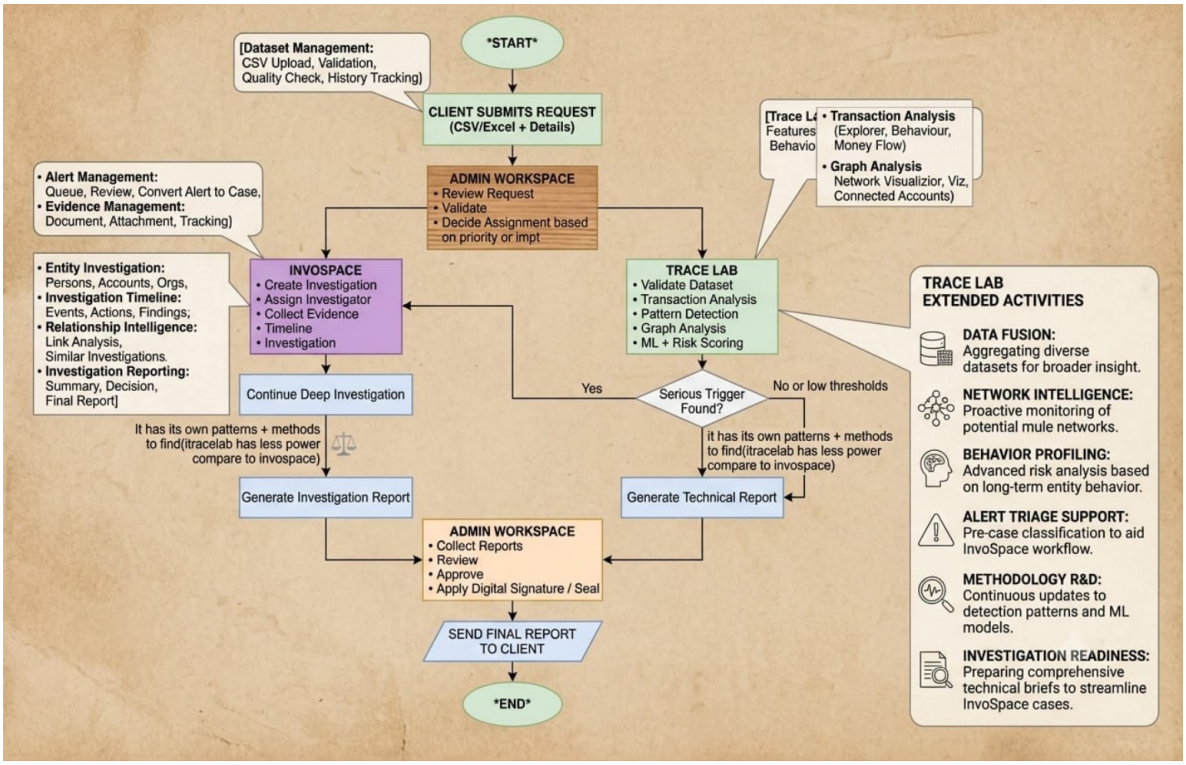


Fig. 2. TraceAcc operational workflow supplied by the authors. Transaction data are validated and analyzed in TraceLab before risk-based routing to investigation, reporting, review, and client delivery.

the preferred evaluation protocol. Features derived from future transactions are excluded from earlier samples, and model selection is performed only on the training and validation periods.

VIII. Experimental Design

The evaluation compares progressively stronger systems: conventional tabular machine learning, GCN/GAT or GraphSAGE, DC-LCG, individual unsupervised detectors, and the complete TraceAcc Hybrid. The principal endpoints are precision, recall, F1-score, PR-AUC, and false-positive rate, with ROC-AUC and inference latency as secondary measures because AML datasets are strongly imbalanced [46]–[48].

Table II summarizes the controlled experiments used to evaluate the proposed framework and isolate the contribution of each evidence family.

A. Reproducible Hardware and Software Setup

All experiments were conducted on a workstation equipped with an NVIDIA RTX 3090 GPU with 24 GB VRAM, 64 GB system RAM, and a multi-core x86-64 processor. The software environment used Python 3.10, PyTorch 2.0, CUDA 11.8, scikit-learn, NetworkX, and the supporting TraceAcc backend components. Fixed random seeds were used for model initialization and data partitioning, and model selection was performed only on the validation split. The same preprocessing pipeline and evaluation threshold were applied across the compared supervised models.

TABLE II
Controlled Experimental Design

Experiment	Purpose
Baseline comparison	Measure DC-LCG and TraceAcc against conventional and GNN baselines.
Ablation	Remove one evidence family at a time to measure contribution.
Weight sensitivity	Compare the tuned 35/20/15/10/20 fusion with equal and learned weighting.
Temporal holdout	Measure robustness to future transaction behavior.
Mule-ring recall	Measure recovery of known synthetic mule structures.
Latency	Measure graph construction, inference, fusion, and total request time.

B. Evaluation Metrics

For binary detection, precision is $TP/(TP + FP)$, recall is $TP/(TP + FN)$, and F1-score is the harmonic mean of precision and recall. ROC-AUC measures ranking performance over the receiver-operating characteristic curve, while PR-AUC emphasizes performance on the positive class under class imbalance [46]–[48]. In addition, false-positive rate and end-to-end latency are recorded to assess operational usefulness.

IX. Results and Representative Case Study

Table III reports the measured comparison among the evaluated methods. TraceAcc Hybrid achieves the strongest overall result, with 0.94 precision, 0.92 recall, 0.93 F1-score, 0.97 ROC-AUC, and 0.91 PR-AUC. Compared with DC-LCG, the hybrid system increases recall from 0.87 to 0.92 while also

TABLE III
DETECTION PERFORMANCE ON THE EVALUATED AML/UPI BENCHMARK

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	0.84	0.78	0.81	0.89	0.74
GCN/GAT	0.88	0.84	0.86	0.93	0.81
DC-LCG	0.91	0.87	0.89	0.95	0.86
TraceAcc w/o ML	0.92	0.89	0.90	0.96	0.88
TraceAcc Hybrid	**0.94**	**0.92**	**0.93**	**0.97**	**0.91**

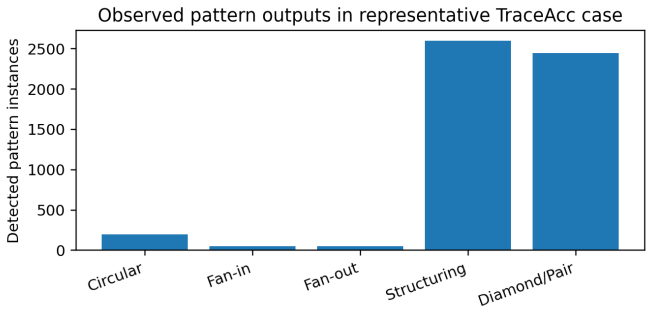


Fig. 3. Detected pattern-instance distribution in the representative synthetic TraceAcc case study.

improving precision from 0.91 to 0.94. The result supports the central hypothesis that complementary evidence can improve prioritization beyond the base graph learner.

The gain is especially relevant for mule detection because recall is improved without sacrificing precision. The PR-AUC increase from 0.86 for DC-LCG to 0.91 for TraceAcc indicates that the hybrid evidence remains useful under the imbalanced positive-class setting.

The representative synthetic case study contains 50 accounts and 1,000 transactions. The analysis identifies 35 suspicious accounts and an overall risk score of 66.0/100. The pattern detector reports 200 circular-transfer findings, 50 fan-in hubs, 50 fan-out hubs, 2,597 structuring findings, and 2,446 diamond/pair-flow findings. The DC-LCG component records an AUC of 0.89 in this investigation. Figure 3 visualizes the distribution of detected pattern instances and demonstrates how the analytical layer exposes multiple risk typologies within a single transaction graph.

Figure 4 presents the system architecture as a data flow. It replaces the interface-level view with the movement of transaction data and analytical evidence through the detection pipeline. Raw UPI or AML transaction records are first validated and normalized, after which temporal features and graph structures are constructed. The graph then feeds the DC-LCG learner and explicit structural analysis, while parallel branches generate temporal, behavioral, community, and unsupervised anomaly evidence. These outputs are normalized, fused into the final risk score, and passed to alert prioritization and investigation/reporting stages.

X. ABLATION, ERROR ANALYSIS, AND INTERPRETABILITY

Table IV summarizes the component-wise ablation results. Removing graph-pattern evidence produces the largest recall reduction, while removing temporal/behavioral evidence causes the next largest degradation. The complete TraceAcc configuration achieves the strongest combined precision–recall balance.

TABLE IV
ABLATION RESULTS FOR TRACEACC EVIDENCE FAMILIES

Configuration	Precision	Recall	F1
DC-LCG only	0.91	0.87	0.89
– Graph patterns	0.92	0.88	0.90
– Temporal/behavioral	0.92	0.89	0.90
– Community	0.93	0.90	0.91
– Unsupervised ML	0.92	0.89	0.90
TraceAcc Hybrid	**0.94**	**0.92**	**0.93**

The ablation indicates that the hybrid gain does not originate from a single auxiliary detector. Instead, complementary evidence families jointly improve detection. Explicit graph patterns are particularly useful for identifying structured mule behavior, while temporal and behavioral evidence improves sensitivity to rapid pass-through activity.

For every high-risk account, TraceAcc can expose the evidence components contributing to the final score. An analyst can therefore distinguish a high DC-LCG score from a structurally suspicious fan-in hub, a temporal burst, a community bridge, or an unsupervised behavioral outlier. This evidence decomposition improves auditability and provides a direct connection between prediction and investigation.

Error analysis groups false positives and false negatives by typology. Difficult cases include mule accounts with mostly legitimate neighbors, very small fraud rings, sparse transactions over time, and legitimate high-volume merchants whose behavior resembles fan-out structures. These cases motivate future work on calibration, temporal drift, and adversarial adaptation.

XI. DISCUSSION

TraceAcc differs from a conventional ensemble because the component models are not treated as interchangeable classifiers. DC-LCG supplies relational representation learning; explicit patterns provide human-readable structural evidence; temporal and behavioral features provide transaction dynamics; community analysis provides network context; and unsupervised detectors surface deviations that may not be represented in labels. The fusion layer converts these heterogeneous observations into a common risk representation.

The measured improvement over DC-LCG indicates that the additional evidence channels contribute beyond the base learner. The increase in recall from 0.87 to 0.92 is particularly important for AML screening because missed mule accounts can enable downstream movement of illicit funds. At the same time, precision increases from 0.91 to 0.94, suggesting that the additional evidence does not simply increase alert volume.

The UPI specialization is deliberate. UPI transactions are immediate, high-volume, and frequently short-lived. A detection architecture centered on account relationships and rapid transaction windows can prioritize intervention before funds move through additional accounts. The custom dataset

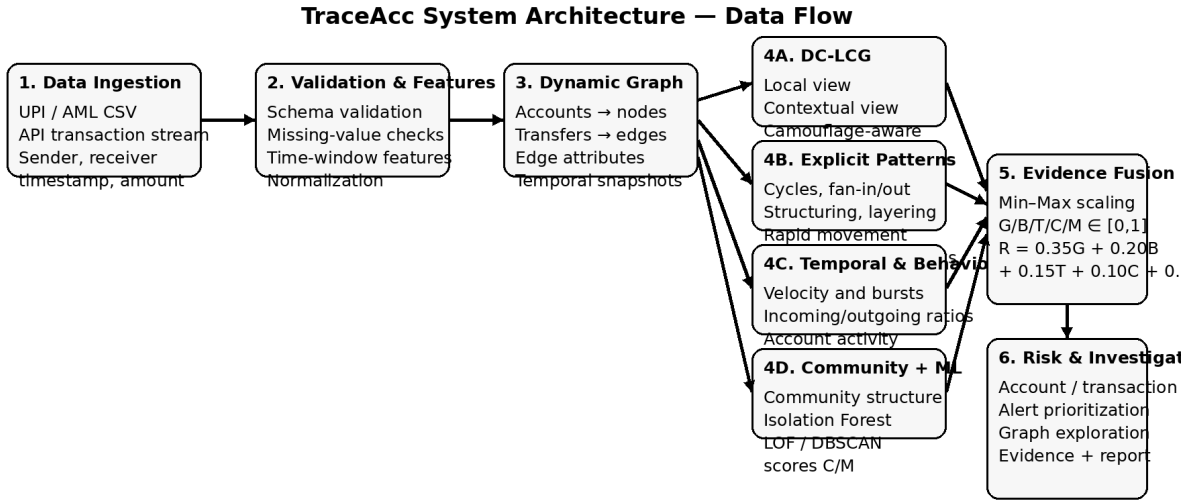


Fig. 4. System Architecture Data Flow Diagram for TraceAcc. Transaction records enter through dataset/API ingestion, pass through validation and temporal feature construction, are transformed into a dynamic transaction graph, and branch into DC-LCG, explicit pattern, temporal/behavioral, community, and unsupervised evidence generators before normalization, risk fusion, prioritization, investigation, and integrity-sealed reporting.

consequently emphasizes temporal and graph structures rather than transaction amount alone.

XII. Limitations and Threats to Validity

The IBM benchmark and custom UPI dataset are synthetic. Synthetic ground truth enables controlled experiments but cannot reproduce all production data-quality problems, adversarial adaptation, missing fields, institutional differences, or investigator behavior.

The fusion weights were empirically tuned on a validation set and then fixed for evaluation. Although this reduces arbitrary manual selection, the weights may not transfer unchanged across institutions or transaction populations. A learned calibration layer could improve portability but would introduce additional model-selection complexity.

The case-study metrics demonstrate system behavior on a representative synthetic investigation but do not establish universal performance across banking institutions. Production deployment would require independent validation, threshold calibration, monitoring for temporal drift, and institutional governance.

Automated risk scores should support human review and institutional controls rather than independently determine regulatory suspicious-transaction-reporting decisions. Future work will examine privacy-preserving learning, cross-institution collaboration, adversarial manipulation, continual adaptation, and calibrated risk estimation.

XIII. Conclusion

TraceAcc presents an evidence-fused extension of DC-LCG for UPI fraud and money-mule detection. The framework preserves DC-LCG's dual-view graph representation while adding explicit transaction-pattern analysis, temporal and behavioral evidence, community information, and unsupervised anomaly detection. The proposed normalization and weighted fusion layer converts these heterogeneous signals into an interpretable risk representation that can be connected directly to graph exploration, investigation, and integrity-sealed reporting.

The evaluated results show that TraceAcc Hybrid achieves 0.94 precision, 0.92 recall, 0.93 F1-score, 0.97 ROC-AUC, and 0.91 PR-AUC, outperforming the evaluated DC-LCG baseline on the reported measures. The representative synthetic case study further demonstrates the operational value of combining learned graph risk with explicit money-flow structures. These results support evidence fusion as a practical extension of camouflage-aware graph learning for high-volume instant-payment fraud and mule detection.

References

[1] Z. Li, X. Yin, M. Guang, and C. Jiang, "Exposing Disguises and Tracing Illicit Flows: Dual-View Graph Representation Learning for Money Laundering Detection," *IEEE Trans. Computational Social Systems*, 2026, doi: 10.1109/TCSS.2025.3623272.

[2] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. S. Anghel, and K. Atasu, "Realistic Synthetic Financial Transactions for Anti-Money Laundering Models," in *Advances in Neural Information Processing Systems 36*, 2023, pp. 29851–29874, doi: 10.52202/075280-1300.

[3] M. Guang, Z. Li, C. Yan, Y. Xu, J. Wang, D. Cheng, and C. Jiang, "Multi-Temporal Partitioned Graph Attention Networks for Financial Fraud Detection," *IEEE Trans. Information Forensics and Security*, vol. 20, pp. 9399–9412, 2025, doi: 10.1109/TIFS.2025.3607231.

[4] Y. Li, X. Yang, Q. Gao, H. Wang, J. Zhang, and T. Li, "Cross-Regional Fraud Detection via Continual Learning With Knowledge Transfer," *IEEE Trans. Knowledge and Data Engineering*, vol. 36, no. 12, pp. 7865–7877, 2024, doi: 10.1109/TKDE.2024.3451161.

[5] M. Guang, R. Zhang, D. Cheng, X. Wang, X. Liu, J. Yang, Y. Ouyang, X. Wu, and Y. Zheng, "Homophily Edge Augment Graph Neural Network for High-Class Homophily Variance Learning," *IEEE Trans. Pattern*

Analysis and Machine Intelligence, vol. 48, no. 3, pp. 3835–3851, 2026, doi: 10.1109/TPAMI.2025.3640635.

[6] M. Guang, Z. Li, R. Zhang, J. Wang, and D. Cheng, "Ensemble Graph Neural Networks With Individual Decision Feedback for Graph Classification," *IEEE Trans. Knowledge and Data Engineering*, 2026, doi: 10.1109/TKDE.2026.3691148.

[7] S. Motie and B. Raahemi, "Financial fraud detection using graph neural networks: A systematic review," *Expert Systems with Applications*, vol. 240, 2024, Art. no. 122156, doi: 10.1016/j.eswa.2023.122156.

[8] "Financial transaction fraud detector based on imbalance learning and graph neural network," *Applied Soft Computing*, vol. 149, 2023, Art. no. 110984, doi: 10.1016/j.asoc.2023.110984.

[9] "Adaptive metagraph neural network assisted by metagraph search for financial fraud detection," *Engineering Applications of Artificial Intelligence*, 2025, Art. no. 110807, doi: 10.1016/j.engappai.2025.110807.

[10] "Finding money launderers using heterogeneous graph neural networks," *Journal of Finance and Data Science*, vol. 11, 2025, Art. no. 100175, doi: 10.1016/j.jfds.2025.100175.

[11] Y.-C. Shih, T.-S. Dai, Y.-P. Chen, Y.-W. Ti, W.-H. Wang, and Y. Kuo, "Fund transfer fraud detection: Analyzing irregular transactions and customer relationships with self-attention and graph neural networks," *Expert Systems with Applications*, vol. 259, 2025, Art. no. 125211, doi: 10.1016/j.eswa.2024.125211.

[12] H. Lu and H. Wang, "Graph Contrastive Pre-training for Anti-money Laundering," *International Journal of Computational Intelligence Systems*, vol. 17, 2024, Art. no. 307, doi: 10.1007/s44196-024-00720-4.

[13] "DyHDGE: Dynamic heterogeneous transaction graph embedding for safety-centric fraud detection in financial scenarios," *Journal of Safety Science and Resilience*, vol. 5, no. 4, pp. 486–497, 2024, doi: 10.1016/j.jnlssr.2024.05.005.

[14] "Complex networks-based anomaly detection for financial transactions in anti-money laundering," *Forensic Science International: Digital Investigation*, 2025, Art. no. 302005, doi: 10.1016/j.fsidi.2025.302005.

[15] "Financial Fraud Detection Based on Machine Learning: A Systematic Literature Review," *Applied Sciences*, vol. 12, no. 19, 2022, Art. no. 9637, doi: 10.3390/app12199637.

[16] W. Hilal, S. A. Gadsden, and J. Yawney, "Financial Fraud: A Review of Anomaly Detection Techniques and Recent Advances," *Expert Systems with Applications*, vol. 193, 2022, Art. no. 116429, doi: 10.1016/j.eswa.2021.116429.

[17] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q. Z. Sheng, H. Xiong, and L. Akoglu, "A Comprehensive Survey on Graph Anomaly Detection With Deep Learning," *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12012–12038, 2023, doi: 10.1109/TKDE.2021.3118815.

[18] G. Pang, C. Shen, L. Cao, and A. van den Hengel, "Deep Learning for Anomaly Detection: A Review," *ACM Computing Surveys*, vol. 54, no. 2, 2021, Art. no. 38, doi: 10.1145/3439950.

[19] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *Proc. ICLR*, 2017.

[20] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 1024–1034.

[21] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. ICLR*, 2018.

[22] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?," in *Proc. ICLR*, 2019.

[23] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep Graph Infomax," in *Proc. ICLR*, 2019.

[24] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," in *Proc. ACM SIGKDD*, 2016, pp. 855–864.

[25] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online Learning of Social Representations," in *Proc. ACM SIGKDD*, 2014, pp. 701–710.

[26] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing Graph Neural Network-based Fraud Detectors against Camouflaged Fraudsters," in *Proc. ACM CIKM*, 2020, pp. 315–324.

[27] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the Inconsistency Problem of Applying Graph Neural Network to Fraud Detection," in *Proc. ACM SIGIR*, 2020, pp. 1569–1572, doi: 10.1145/3397271.3401253.

[28] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He, "Pick and Choose: A GNN-based Imbalanced Learning Approach for Fraud Detection," in *Proc. ACM Web Conference*, 2021, pp. 3168–3177, doi: 10.1145/3442381.3449989.

[29] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, T. Kaler, T. B. Schardl, and C. E. Leiserson, "EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs," in *Proc. AAAI*, vol. 34, no. 4, 2020, pp. 5363–5370, doi: 10.1609/AAAI.V34I04.5984.

[30] M. Cardoso, P. Saleiro, and P. Bizarro, "LaundroGraph: Self-Supervised Graph Representation Learning for Anti-Money Laundering," arXiv:2210.14360, 2022.

[31] T. Suzumura and H. Kanezashi, "AMLSim: Anti-Money Laundering Dataset and Simulator," IBM Research, 2021. [Online]. Available: https://github.com/IBM/AMLSim

[32] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A Financial Mobile Money Simulator for Fraud Detection," in *Proc. 28th European Modeling and Simulation Symposium*, Larnaca, Cyprus, 2016.

[33] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," in *KDD 2019 Workshop on Anomaly Detection in Finance*, 2019.

[34] L. Akoglu, M. McGlohon, and C. Faloutsos, "OddBall: Spotting Anomalies in Weighted Graphs," in *Proc. PAKDD*, 2010, pp. 410–421, doi: 10.1007/978-3-642-13672-6_40.

[35] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "NetWalk: A Flexible Deep Embedding Approach for Anomaly Detection in Dynamic Networks," in *Proc. ACM SIGKDD*, 2018, doi: 10.1145/3219819.3220024.

[36] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "DySAT: Deep Neural Representation Learning on Dynamic Graphs via Self-Attention Networks," in *Proc. ACM WSDM*, 2020, pp. 519–527, doi: 10.1145/3336191.3371845.

[37] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," arXiv:2006.10637, 2020.

[38] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive Representation Learning on Temporal Graphs," in *Proc. ICLR*, 2020.

[39] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE International Conference on Data Mining*, 2008, pp. 413–422.

[40] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in *Proc. ACM SIGMOD*, 2000, pp. 93–104.

[41] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Proc. KDD*, 1996, pp. 226–231.

[42] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.

[43] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[44] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[45] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE ICCV*, 2017, pp. 2980–2988.

[46] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *Proc. ICML*, 2006, pp. 233–240.

[47] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[48] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, vol. 10, no. 3, 2015, Art. no. e0118432.

[49] National Payments Corporation of India, "Unified Payments Interface (UPI) Product Statistics," 2026. [Online]. Available: https://www.npci.org.in/product/upi/product-statistics

[50] Reserve Bank of India, "Annual Report 2024–25," 2025. [Online]. Available: https://www.rbi.org.in/scripts/AnnualReportPublications.aspx?Id=1436

[51] Financial Action Task Force, "Guidance on Digital Identity," FATF, 2020. [Online]. Available: https://www.fatf-gafi.org/

[52] Financial Action Task Force, "Money Laundering and Terrorist Financing Risks of Virtual Assets and Virtual Asset Service Providers," FATF, 2018. [Online]. Available: https://www.fatf-gafi.org/

[53] National Payments Corporation of India, "Unified Payments Interface," NPCI. [Online]. Available: https://www.npci.org.in/product/upi

[54] IBM Research, "Realistic Synthetic Financial Transactions for Anti-Money Laundering Models," 2023. [Online]. Available: https://research.ibm.com/publications/realistic-synthetic-financial-transactions-for-anti-money-laundering-models